

DBMS TASK

ZARA FASHION E-COMMERCE MANAGEMENT SYSTEM

Review and rating

1.Database design

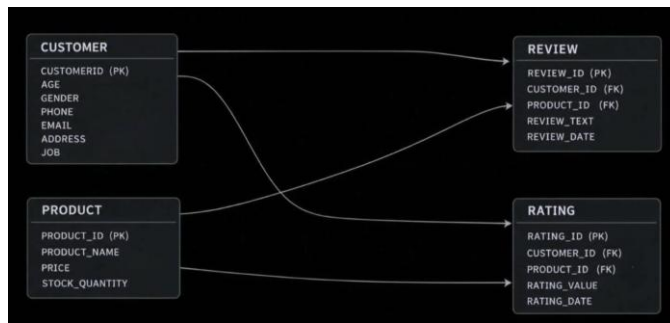
1.1 Review table

Attribute	Data Type	Constraint
REVIEW_ID	NUMBER(5)	PRIMARY KEY
CUSTOMER_ID	NUMBER(5)	NOT NULL
PRODUCT_ID	NUMBER(5)	NOT NULL
REVIEW_TEXT	VARCHAR2(200)	NOT NULL
REVIEW_DATE	DATE	NOT NULL

1.2 Rating table

Attribute	Data Type	Constraint
RATING_ID	NUMBER(5)	PRIMARY KEY
CUSTOMER_ID	NUMBER(5)	NOT NULL
PRODUCT_ID	NUMBER(5)	NOT NULL
RATING_VALUE	NUMBER(1)	NOT NULL
RATING_DATE	DATE	NOT NULL

2. Table Relationships



3. Table Creation

3.1 Create Review table

```
SQL> CREATE TABLE Review
2  (
3      REVIEW_ID NUMBER(5) PRIMARY KEY,
4      CUSTOMER_ID NUMBER(5) NOT NULL,
5      PRODUCT_ID NUMBER(5) NOT NULL,
6      REVIEW_TEXT VARCHAR2(200),
7      REVIEW_DATE DATE NOT NULL
8  );
```

Table created.

3.2 Create Rating table

```
SQL> CREATE TABLE Rating
2  (
3      RATING_ID NUMBER(5) PRIMARY KEY,
4      CUSTOMER_ID NUMBER(5) NOT NULL,
5      PRODUCT_ID NUMBER(5) NOT NULL,
6      RATING_VALUE NUMBER(1) NOT NULL,
7      RATING_DATE DATE NOT NULL
8  );
```

Table created.

4. Data insertion

4.1 Review data insertion

```

SQL> INSERT INTO Review
  2 (REVIEW_ID, CUSTOMER_ID, PRODUCT_ID, REVIEW_TEXT, REVIEW_DATE)
  3 VALUES
  4 (1, 108, 101, 'Good quality product', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (REVIEW_ID, CUSTOMER_ID, PRODUCT_ID, REVIEW_TEXT, REVIEW_DATE)
  3 VALUES
  4 (2, 224, 102, 'Nice and comfortable', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (REVIEW_ID, CUSTOMER_ID, PRODUCT_ID, REVIEW_TEXT, REVIEW_DATE)
  3 VALUES
  4 (3, 567, 103, 'Good fitting and quality', SYSDATE);

1 row created.

```

4.2 Rating Data insertion

```

SQL> INSERT INTO Rating
  2 (RATING_ID, CUSTOMER_ID, PRODUCT_ID, RATING_VALUE, RATING_DATE)
  3 VALUES
  4 (2, 224, 102, 4, SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Rating
  2 (RATING_ID, CUSTOMER_ID, PRODUCT_ID, RATING_VALUE, RATING_DATE)
  3 VALUES
  4 (3, 567, 103, 5, SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Rating
  2 (RATING_ID, CUSTOMER_ID, PRODUCT_ID, RATING_VALUE, RATING_DATE)
  3 VALUES
  4 (4, 108, 104, 3, SYSDATE);

1 row created.

```

5.Retrieve product review details :

```
SQL> SELECT R.REVIEW_ID,
2         R.CUSTOMER_ID,
3         R.PRODUCT_ID,
4         R.REVIEW_TEXT,
5         R.REVIEW_DATE
6 FROM Review R
7 ORDER BY R.PRODUCT_ID;
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
```

```
REVIEW_DA
-----
          1          108          101
Good quality product
06-OCT-26

          2          224          102
Nice and comfortable
06-OCT-26
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
```

```
REVIEW_DA
-----
          3          567          103
Good fitting and quality
06-OCT-26

          4          108          104
Product is good
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
```

6. Calculate Average Product Ratings

```
SQL> SELECT PRODUCT_ID,
2         AVG(RATING_VALUE) AS AVERAGE_RATING
3 FROM Rating
4 GROUP BY PRODUCT_ID;
```

```
PRODUCT_ID AVERAGE_RATING
-----
          101              5
          102              4
          103              5
          104              3
          105              4
```

7. Identify Highly Rated Products

```
SQL> SELECT PRODUCT_ID,
2         AVG(RATING_VALUE) AS AVERAGE_RATING
3 FROM Rating
4 GROUP BY PRODUCT_ID
5 HAVING AVG(RATING_VALUE) >= 4;
```

PRODUCT_ID	AVERAGE_RATING
101	5
102	4
103	5
105	4

8. Product rating report

```
SQL> SELECT PRODUCT_ID,
2         COUNT(RATING_ID) AS TOTAL_RATINGS,
3         AVG(RATING_VALUE) AS AVERAGE_RATING,
4         MAX(RATING_VALUE) AS HIGHEST_RATING,
5         MIN(RATING_VALUE) AS LOWEST_RATING
6 FROM Rating
7 GROUP BY PRODUCT_ID
8 ORDER BY PRODUCT_ID;
```

PRODUCT_ID	TOTAL_RATINGS	AVERAGE_RATING	HIGHEST_RATING	LOWEST_RATING
101	1	5	5	5
102	1	4	4	4
103	1	5	5	5
104	1	3	3	3
105	1	4	4	4